JavaScript Beginner Class: Variables, Operators, Conditions, Loops, Functions, and Events

Class Goal

By the end of this class, students will understand the basic building blocks of JavaScript and be able to write simple interactive programs for the browser.

Students will learn:

1. What variables are and how to use them
2. How operators work
3. How to make decisions using conditions
4. How to repeat actions using loops
5. How to organize code using functions
6. How to make web pages interactive using events

---

1. JavaScript Variables

What is a variable?

A variable is like a box where we store data. The data can be a name, number, true/false value, or something else.

For example, if we want to store a student's name, we can use a variable.

```
let studentName = "Rahim";
console.log(studentName);
```

Output:

```
Rahim
```

Here:

- `let` is used to create a variable.
- `studentName` is the variable name.
- `"Rahim"` is the value stored inside the variable.
- `console.log()` shows the value in the browser console.

Types of variables

JavaScript has three main ways to create variables:

```
var name = "Emon";
let age = 20;
const country = "Bangladesh";
```

`var`

`var` is an older way to create variables. It still works, but modern JavaScript usually uses `let` and `const`.

```
var city = "Dhaka";
console.log(city);
```

`let`

Use `let` when the value may change later.

```
let score = 10;
score = 20;
console.log(score);
```

Output:

```
20
```

`const`

Use `const` when the value should not change.

```
const birthYear = 2005;
console.log(birthYear);
```

If you try to change a `const` value, JavaScript will show an error.

```
const country = "Bangladesh";
country = "India"; // Error
```

Common data types

String

A string is text. It is written inside quotes.

```
let name = "Karim";
let message = "Welcome to JavaScript!";
```

Number

A number can be used for math.

```
let age = 18;
let price = 99.5;
```

Boolean

A boolean means `true` or `false`.

```
let isStudent = true;
let isLoggedIn = false;
```

Variable naming rules

Good variable names should be clear and meaningful.

Good examples:

```
let userName = "Sadia";
let totalPrice = 500;
let isActive = true;
```

Bad examples:

```
let x = "Sadia";
let tp = 500;
let a = true;
```

These work, but they are hard to understand.

Instructions for students

When creating variables:

1. Use `let` if the value can change.
2. Use `const` if the value should not change.
3. Use meaningful names.
4. Use camelCase for variable names.

Example of camelCase:

```
let firstName = "Nadia";
let totalMarks = 85;
```

---

2. JavaScript Operators

What is an operator?

An operator is a symbol that performs an action.

For example, `+` adds numbers.

```
let result = 10 + 5;
console.log(result);
```

Output:

```
15
```

Arithmetic operators

Arithmetic operators are used for math.

Operator	Meaning	Example
`+`	Addition	`10 + 5`
`-`	Subtraction	`10 - 5`
`*`	Multiplication	`10 * 5`

Operator	Meaning	Example
`/`	Division	`10 / 5`
`%`	Remainder	`10 % 3`

Examples:

```
let a = 10;
let b = 3;

console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.3333333333333335
console.log(a % b); // 1
```

Assignment operators

Assignment operators are used to assign or update values.

```
let score = 10;
score += 5;
console.log(score);
```

Output:

```
15
```

Common assignment operators:

Operator	Example	Same as
`=`	`x = 10`	Assign 10 to x
`+=`	`x += 5`	`x = x + 5`
`-=`	`x -= 5`	`x = x - 5`
`*=`	`x *= 5`	`x = x * 5`
`/=`	`x /= 5`	`x = x / 5`

Comparison operators

Comparison operators compare two values. The result is usually `true` or `false`.

```
let age = 18;
console.log(age >= 18);
```

Output:

```
true
```

Common comparison operators:

Operator	Meaning	Example
`==`	Equal value	`5 == "5"`
`===`	Equal value and type	`5 === "5"`
`!=`	Not equal value	`5 != 3`
`!==`	Not equal value or type	`5 !== "5"`
`>`	Greater than	`10 > 5`
`<`	Less than	`10 < 5`
`>=`	Greater than or equal	`10 >= 10`
`<=`	Less than or equal	`5 <= 10`

Important example:

```
console.log(5 == "5");  // true
console.log(5 === "5"); // false
```

Explanation:

- `==` checks only value.
- `===` checks value and data type.

In modern JavaScript, use `===` most of the time.

Logical operators

Logical operators are used when we need more than one condition.

Operator	Meaning
`&&`	AND
`||`	OR
`!`	NOT

AND operator

Both conditions must be true.

```
let age = 20;
let hasTicket = true;

console.log(age >= 18 && hasTicket === true);
```

Output:

```
true
```

OR operator

At least one condition must be true.

```
let hasEmail = false;
let hasPhone = true;

console.log(hasEmail || hasPhone);
```

Output:

```
true
```

NOT operator

The NOT operator reverses the value.

```
let isLoggedIn = false;
console.log(!isLoggedIn);
```

Output:

```
true
```

---

3. JavaScript Conditions

What is a condition?

A condition allows JavaScript to make decisions.

Example from real life:

If it is raining, take an umbrella. Otherwise, go without an umbrella.

In JavaScript:

```
let isRaining = true;

if (isRaining) {
  console.log("Take an umbrella");
} else {
  console.log("No umbrella needed");
}
```

Output:

```
Take an umbrella
```

`if` statement

Use `if` when you want code to run only when a condition is true.

```
let age = 18;

if (age >= 18) {
  console.log("You are an adult");
}
```

`if...else` statement

Use `else` when you want another block of code to run if the condition is false.

```
let age = 16;

if (age >= 18) {
  console.log("You can vote");
} else {
  console.log("You cannot vote yet");
}
```

Output:

```
You cannot vote yet
```

`if...else if...else`

Use this when you have multiple conditions.

```
let marks = 75;

if (marks >= 80) {
  console.log("Grade: A+");
} else if (marks >= 70) {
  console.log("Grade: A");
} else if (marks >= 60) {
  console.log("Grade: B");
} else {
  console.log("Grade: F");
}
```

Output:

```
Grade: A
```

Switch statement

A `switch` statement is useful when checking one value against many possible choices.

```
let day = "Friday";

switch (day) {
  case "Friday":
    console.log("Today is holiday");
    break;
  case "Saturday":
    console.log("Weekend class day");
    break;
  default:
    console.log("Regular day");
}
```

Output:

```
Today is holiday
```

Important: Use `break` to stop the switch after a matching case.

---

4. JavaScript Loops

What is a loop?

A loop repeats code again and again.

Example:

Instead of writing this:

```
console.log("Hello");
console.log("Hello");
console.log("Hello");
```

We can write:

```
for (let i = 1; i <= 3; i++) {
  console.log("Hello");
}
```

Output:

```
Hello
Hello
Hello
```

`for` loop

A `for` loop is useful when you know how many times you want to repeat something.

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

Output:

```
1
2
3
4
5
```

Explanation:

```
for (let i = 1; i <= 5; i++)
```

- `let i = 1` starts the counter.
- `i <= 5` keeps the loop running while this is true.
- `i++` increases `i` by 1 after each loop.

`while` loop

A `while` loop runs while a condition is true.

```
let count = 1;

while (count <= 5) {
  console.log(count);
  count++;
}
```

Output:

```
1
2
3
4
5
```

`do...while` loop

A `do...while` loop runs at least once, even if the condition is false.

```
let number = 10;

do {
  console.log(number);
  number++;
} while (number <= 5);
```

Output:

```
10
```

Why? Because the code runs first, then checks the condition.

Looping through an array

An array stores multiple values.

```
let students = ["Rahim", "Karim", "Sadia"];

for (let i = 0; i < students.length; i++) {
  console.log(students[i]);
}
```

Output:

```
Rahim
Karim
Sadia
```

`for...of` loop

This is an easier way to loop through an array.

```
let fruits = ["Mango", "Banana", "Apple"];

for (let fruit of fruits) {
  console.log(fruit);
}
```

Output:

```
Mango
Banana
Apple
```

---

5. JavaScript Functions

What is a function?

A function is a reusable block of code.

Instead of writing the same code many times, we can write it once and use it whenever needed.

```
function sayHello() {
  console.log("Hello, welcome to JavaScript!");
}

sayHello();
sayHello();
```

Output:

```
Hello, welcome to JavaScript!
Hello, welcome to JavaScript!
```

Function with parameters

Parameters allow us to send values into a function.

```
function greetUser(name) {
  console.log("Hello " + name);
}

greetUser("Rahim");
greetUser("Sadia");
```

Output:

```
Hello Rahim
Hello Sadia
```

Here, `name` is a parameter.

Function with return value

A function can return a result.

```
function addNumbers(a, b) {
  return a + b;
}

let result = addNumbers(10, 5);
console.log(result);
```

Output:

```
15
```

Function expression

A function can also be stored inside a variable.

```
const multiply = function(a, b) {
  return a * b;
};

console.log(multiply(4, 5));
```

Output:

```
20
```

Arrow function

Arrow functions are a shorter way to write functions.

```
const subtract = (a, b) => {
  return a - b;
};

console.log(subtract(10, 3));
```

Output:

```
7
```

Shorter version:

```
const square = number => number * number;

console.log(square(5));
```

Output:

```
25
```

Instructions for students

Use functions when:

1. You need to reuse code.
2. Your code is getting too long.
3. You want your program to be easier to read.
4. You want to separate tasks into smaller parts.

---

6. JavaScript Events

What is an event?

An event is something that happens on a web page.

Examples of events:

- A user clicks a button
- A user types in an input box
- A user moves the mouse
- A page finishes loading

JavaScript can listen for events and respond to them.

Example HTML

```
<button id="myButton">Click Me</button>
<p id="message"></p>
```

Click event example

```
let button = document.getElementById("myButton");
let message = document.getElementById("message");

button.addEventListener("click", function() {
  message.textContent = "Button clicked!";
});
```

When the user clicks the button, the text changes to `Button clicked!`.

Input event example

HTML:

```
<input id="nameInput" type="text" placeholder="Enter your name">
<p id="output"></p>
```

JavaScript:

```
let input = document.getElementById("nameInput");
let output = document.getElementById("output");

input.addEventListener("input", function() {
  output.textContent = "Hello " + input.value;
});
```

When the user types a name, the page shows a greeting.

Mouse event example

HTML:

```
<button id="hoverButton">Hover over me</button>
```

JavaScript:

```
let hoverButton = document.getElementById("hoverButton");

hoverButton.addEventListener("mouseover", function() {
  console.log("Mouse is over the button");
});
```

Common event types

Event	Meaning
`click`	When an element is clicked
`input`	When the user types in an input field
`mouseover`	When the mouse moves over an element
`mouseout`	When the mouse leaves an element
`keydown`	When a keyboard key is pressed
`submit`	When a form is submitted

Full mini project example: Simple Greeting App

HTML:

```
<input id="username" type="text" placeholder="Enter your name">
<button id="greetButton">Greet Me</button>
<p id="greeting"></p>
```

JavaScript:

```
let username = document.getElementById("username");
let greetButton = document.getElementById("greetButton");
let greeting = document.getElementById("greeting");

greetButton.addEventListener("click", function() {
  greeting.textContent = "Welcome, " + username.value + "!";
});
```

How it works:

1. The user types a name.
2. The user clicks the button.
3. JavaScript reads the input value.
4. JavaScript shows a greeting message.

Student Tasks

Task 1: Variables

Create variables for:

1. Your name
2. Your age
3. Your city
4. Whether you are a student

Then print them using `console.log()`.

Example:

```
let name = "Emon";
let age = 22;
let city = "Dhaka";
let isStudent = true;

console.log(name);
```

```
console.log(age);
console.log(city);
console.log(isStudent);
```

Task 2: Operators

Create two numbers and show:

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Remainder

Example:

```
let num1 = 20;
let num2 = 6;

console.log(num1 + num2);
console.log(num1 - num2);
console.log(num1 * num2);
console.log(num1 / num2);
console.log(num1 % num2);
```

Task 3: Conditions

Write a program that checks if a student passed or failed.

Rule:

- If marks are 33 or more, show `Passed`.
- Otherwise, show `Failed`.

```
let marks = 40;

if (marks >= 33) {
  console.log("Passed");
} else {
  console.log("Failed");
}
```

Task 4: Loops

Print numbers from 1 to 10 using a `for` loop.

```
for (let i = 1; i <= 10; i++) {
  console.log(i);
}
```

Task 5: Functions

Create a function that takes two numbers and returns their total.

```
function total(a, b) {
  return a + b;
}

console.log(total(10, 20));
```

Task 6: Events

Create a button. When the button is clicked, show the text `Hello JavaScript` on the page.

HTML:

```
<button id="showButton">Show Message</button>
<p id="message"></p>
```

JavaScript:

```
let showButton = document.getElementById("showButton");
let message = document.getElementById("message");

showButton.addEventListener("click", function() {
  message.textContent = "Hello JavaScript";
});
```

Common Student Mistakes

Mistake 1: Forgetting quotes around strings

Wrong:

```
let name = Rahim;
```

Correct:

```
let name = "Rahim";
```

Explanation: Text must be inside quotes.

Mistake 2: Using `=` instead of `===`

Wrong:

```
if (age = 18) {
  console.log("Adult");
}
```

Correct:

```
if (age === 18) {
  console.log("Adult");
}
```

Explanation:

- `=` assigns a value.
- `===` compares values.

Mistake 3: Forgetting curly braces

Wrong:

```
if (age >= 18)
  console.log("Adult");
  console.log("Welcome");
```

Correct:

```
if (age >= 18) {
  console.log("Adult");
  console.log("Welcome");
}
```

Explanation: Curly braces group code together.

Mistake 4: Infinite loop

Wrong:

```
let i = 1;

while (i <= 5) {
  console.log(i);
}
```

Correct:

```
let i = 1;

while (i <= 5) {
  console.log(i);
  i++;
}
```

Explanation: Always update the loop counter.

Mistake 5: Calling a function before understanding parameters

Wrong:

```
function greet(name) {
  console.log("Hello " + name);
}
```

```
greet();
```

Output:

```
Hello undefined
```

Correct:

```
greet("Rahim");
```

Explanation: The function needs a value for `name`.

Mistake 6: JavaScript file loading before HTML elements

Problem:

```
let button = document.getElementById("myButton");
```

If JavaScript runs before the button exists in HTML, `button` may be `null`.

Solution 1: Put the script before the closing `</body>` tag.

```
<body>
  <button id="myButton">Click</button>

  <script src="script.js"></script>
</body>
```

Solution 2: Use `DOMContentLoaded`.

```
document.addEventListener("DOMContentLoaded", function() {
  let button = document.getElementById("myButton");
});
```

FAQ

1. What is JavaScript used for?

JavaScript is used to make websites interactive. It can respond to clicks, show messages, validate forms, create animations, fetch data, and much more.

2. What is the difference between HTML, CSS, and JavaScript?

HTML creates the structure of a web page.

CSS styles the page.

JavaScript adds behavior and interactivity.

Example:

- HTML: Button exists
- CSS: Button looks beautiful
- JavaScript: Button does something when clicked

3. Should I use `var`, `let`, or `const`?

Use `let` when the value may change.

Use `const` when the value should not change.

Avoid `var` when writing modern JavaScript.

4. Why should I use `===` instead of `==`?

`===` checks both value and data type, so it is safer and clearer.

Example:

```
console.log(5 == "5");  // true
console.log(5 === "5"); // false
```

5. What is the difference between a parameter and an argument?

A parameter is the name used inside the function definition.

An argument is the actual value passed when calling the function.

```
function greet(name) { // name is parameter
  console.log("Hello " + name);
}

greet("Rahim"); // Rahim is argument
```

6. Why do loops sometimes never stop?

A loop may never stop if the condition always stays true.

Example:

```
let i = 1;

while (i <= 5) {
  console.log(i);
}
```

Here, `i` is never increased, so the condition always remains true.

7. What is an event listener?

An event listener waits for something to happen, like a button click.

```
button.addEventListener("click", function() {
  console.log("Button clicked");
});
```

8. Do I need to memorize everything?

No. Beginners do not need to memorize everything. The most important thing is to practice writing code and understanding how it works.

Final Class Project

Project: Student Result Checker

Create a small web app where:

1. The user enters marks.
2. The user clicks a button.
3. The app shows whether the student passed or failed.

HTML:

```
<input id="marksInput" type="number" placeholder="Enter marks">
<button id="checkButton">Check Result</button>
<p id="result"></p>
```

JavaScript:

```
let marksInput = document.getElementById("marksInput");
let checkButton = document.getElementById("checkButton");
let result = document.getElementById("result");

checkButton.addEventListener("click", function() {
  let marks = Number(marksInput.value);

  if (marks >= 33) {
    result.textContent = "Passed";
  } else {
    result.textContent = "Failed";
  }
});
```

Extra challenge:

Add grades:

- 80 or above: A+
- 70 to 79: A
- 60 to 69: B
- 50 to 59: C
- 33 to 49: D
- Below 33: F
